Exact Multiplicative Certificates for Empirical Categorical SxPID2

A complete sign reduction, a retained counterexample, and bounded executable refinement evidence

pid-rs project analysis

25 July 2026

Abstract

This note gives a self-contained exact-arithmetic assurance route for the complete averaged two-source empirical categorical shared-exclusions PID encoded by the pid-rs definition revision `makkeh-gutknecht-wibral-2021-empirical-sxpid2-v1`. After the published keyed source-event disjunctions, keyed target intersections, empirical averaging rule, and integer Möbius transform are fixed, every cumulative and atom is exactly $n^{-1} \log R$ for a positive rational product R. Hence its exact zero and sign are decided without a transcendental approximation by comparing R with one.

The first implementation used an empty canonical log-term map as its only exact-zero witness. That witness is sound but incomplete. An exhaustive binary search produced a minimized five-term, total-count-eight counterexample whose exact product is one although its canonical term map is nonempty. The live directed interval correctly straddles zero. The repaired schema keeps that interval decision unresolved and emits a separate bounded exact-product zero certificate. The proof, the failure, the repair, resource preflight, independent verification, Lean kernel check, exhaustive boundary, mutations, and scientific claim boundary are all stated explicitly. No new PID is defined, and no claim about population calibration, continuous PID, or downstream authority follows.

Contents

1 Scope and nonclaims

The mathematical statement is representation-agnostic only after one representation has already been fixed: the Makkeh–Gutknecht–Wibral shared-exclusions event construction, its two-source redundancy lattice, exact empirical probabilities, and its integer Möbius transform. Changing the event logic, lattice, smoothing rule, or weights changes the certified object. Exact arithmetic does not establish that SxPID is the unique, universally preferred, or scientifically adequate PID.

The result applies to one finite empirical count table. It does not prove data authenticity, population support, independence, ergodicity, stationarity, dependence-color premises, drift bounds, estimator consistency, calibrated uncertainty, equivalence of quantized and continuous estimands, continuous KSG or ISX validity, higher-source PID, or downstream operational fitness. It does not prove pid-core binary64 results. It also does not verify the Rust compiler, Python, GMP, MPFR, Rug, hardware, or the operating system.

2 Fixed empirical definition

2.1 Count table and keyed events

Let Z_+ be the finite set of observed complete states $z = (s_1, s_2, t)$. Each state has an integer multiplicity $c_z > 0$, and

$$n = \sum_{z \in Z_+} c_z > 0. \tag{1}$$

Fix a two-source redundancy-lattice node α. For a keyed observed state z, let $A_{z,\alpha}$ be the disjunction of the node's source-collection events, exactly as in the shared-exclusions construction. Define integer masses

$$a_{z,\alpha} = \#A_{z,\alpha}, \tag{2}$$
$$b_{z,\alpha} = \#(A_{z,\alpha} \cap \{T = t_z\}), \tag{3}$$
$$t_z = \#\{T = t_z\}. \tag{4}$$

Every defining event contains its keyed observed state, so

$$0 < c_z \le b_{z,\alpha} \le a_{z,\alpha} \le n, \qquad b_{z,\alpha} \le t_z \le n. \tag{5}$$

Thus every rational logarithm argument below is strictly positive.

2.2 Pointwise and averaged cumulatives

In nats, define

$$i^{+}_{z,\alpha} = \log \frac{n}{a_{z,\alpha}}, \tag{6}$$

$$i^{-}_{z,\alpha} = \log \frac{t_z}{b_{z,\alpha}}, \tag{7}$$

$$i^{\mathrm{sx}}_{z,\alpha} = i^{+}_{z,\alpha} - i^{-}_{z,\alpha} = \log \frac{n b_{z,\alpha}}{a_{z,\alpha} t_z}. \tag{8}$$

For $u \in \{+, -, \mathrm{sx}\}$, the empirical averaged cumulative is

$$C^{u}_{\alpha} = \sum_{z \in Z_+} \frac{c_z}{n} i^{u}_{z,\alpha}. \tag{9}$$

No pseudo-count, smoothing rule, or alternate weighting is silently inserted.

2.3 Two-source atoms

Let $M_{\gamma\alpha} \in \mathbb{Z}$ be the fixed Möbius coefficient mapping cumulatives to an atom γ. The informative, misinformative, and net atoms are

$$\Pi^{u}_{\gamma} = \sum_{\alpha} M_{\gamma\alpha} C^{u}_{\alpha}. \tag{10}$$

Integer coefficients are the essential algebraic property. Negative coefficients correspond to exact reciprocals in the product representation.

3 Exact product normalization

Definition 3.1 (Cumulative products)**.** For each node α, define positive rationals

$$R^{+}_{\alpha} = \prod_{z \in Z_+} \left(\frac{n}{a_{z,\alpha}} \right)^{c_z}, \tag{11}$$

$$R^{-}_{\alpha} = \prod_{z \in Z_+} \left(\frac{t_z}{b_{z,\alpha}} \right)^{c_z}, \tag{12}$$

$$R^{\mathrm{sx}}_{\alpha} = \prod_{z \in Z_+} \left(\frac{n b_{z,\alpha}}{a_{z,\alpha} t_z} \right)^{c_z}. \tag{13}$$

Theorem 3.2 (Exact cumulative normalization)**.** *For every fixed node and component,*

$$C^{u}_{\alpha} = \frac{1}{n} \log R^{u}_{\alpha}. \tag{14}$$

Moreover,

$$R^{\mathrm{sx}}_{\alpha} = \frac{R^{+}_{\alpha}}{R^{-}_{\alpha}}. \tag{15}$$

Proof. Substitute equations (6)–(8) into equation (9). For the informative component,

$$C^{+}_{\alpha} = \frac{1}{n} \sum_{z} c_z \log \frac{n}{a_{z,\alpha}} = \frac{1}{n} \log \prod_{z} \left(\frac{n}{a_{z,\alpha}} \right)^{c_z} = \frac{1}{n} \log R^{+}_{\alpha}.$$

The same logarithm product and integer-power rules prove the misinformative and net identities. Equation (8) gives the local ratio of informative to misinformative arguments. Raising each ratio to c_z, multiplying, and using equations (11)–(13) proves equation (15). □

Definition 3.3 (Atom products)**.** For every atom γ, let

$$R^u_\gamma = \prod_\alpha (R^u_\alpha)^{M_{\gamma\alpha}}. \tag{16}$$

Theorem 3.4 (Exact atom normalization)**.** *Every two-source atom satisfies*

$$\Pi^u_\gamma = \frac{1}{n}\log R^u_\gamma. \tag{17}$$

Proof. Apply Theorem 3.2 in equation (10):

$$\Pi^u_\gamma = \frac{1}{n}\sum_\alpha M_{\gamma\alpha}\log R^u_\alpha = \frac{1}{n}\log\prod_\alpha (R^u_\alpha)^{M_{\gamma\alpha}} = \frac{1}{n}\log R^u_\gamma.$$

The signed integer power rule is valid because every R^u_α is positive. □

Corollary 3.5 (Complete exact sign decision)**.** *For any cumulative or atom represented by a positive rational R,*

$$\frac{1}{n}\log R = 0 \iff R = 1, \tag{18}$$

$$\frac{1}{n}\log R > 0 \iff R > 1, \tag{19}$$

$$\frac{1}{n}\log R < 0 \iff R < 1. \tag{20}$$

Proof. Equation (1) implies $1/n > 0$, so multiplication by $1/n$ preserves order and zero. The natural logarithm is strictly increasing on $(0,\infty)$, and $\log 1 = 0$. Therefore its zero, positive, and negative regions are respectively $R = 1$, $R > 1$, and $0 < R < 1$. □

Remark 3.6 (Canonical log-linear form)**.** If a report coordinate is $V = \sum_j q_j \log x_j$, empirical construction gives $nq_j \in \mathbb{Z}$. Hence

$$V = \frac{1}{n}\log R, \qquad R = \prod_j x_j^{nq_j}. \tag{21}$$

This binds the event-count product and emitted term-list product. It is not a prime-factorization claim.

4 Retained failure: nonempty terms can cancel exactly

Counterexample 4.1 (The empty-term witness is incomplete)**.** Use canonical binary state order

$$(0,0,0),(0,0,1),(0,1,0),(0,1,1),(1,0,0),(1,0,1),(1,1,0),(1,1,1)$$

and counts

$$(0,0,1,1,1,4,1,0), \qquad n = 8. \tag{22}$$

For the net unique-one atom, exact extraction gives the nonempty expression

$$E = -\frac{1}{8}\log\frac{8}{15} + \frac{1}{8}\log\frac{4}{5} + \frac{1}{8}\log\frac{8}{9} + \frac{1}{8}\log\frac{4}{3} - \frac{1}{8}\log\frac{16}{9}. \tag{23}$$

Its denominator-cleared product is

$$\begin{aligned} R &= \left(\frac{8}{15}\right)^{-1}\left(\frac{4}{5}\right)\left(\frac{8}{9}\right)\left(\frac{4}{3}\right)\left(\frac{16}{9}\right)^{-1} \\ &= \frac{15}{8}\frac{4}{5}\frac{8}{9}\frac{4}{3}\frac{9}{16} = 1. \end{aligned} \tag{24}$$

Therefore $E = \frac{1}{8}\log R = 0$, despite five retained canonical terms.

At 256 working bits, the live directed interval is

$$[-3\,2^{-258},\ 5\,2^{-259}]. \tag{25}$$

It correctly contains zero and correctly fails to determine a strict sign. The old rule "exact zero only if the canonical map is empty" therefore missed a true exact zero. Replacing the interval decision by exact zero would also be wrong, because the interval endpoints do not prove it. Report schema v2 preserves both truths:

- interval decision: `unresolved_sign`, interval zero witness absent;
- exact-product decision: `certified_exact_zero`;
- exact-product witness: `exact_multiplicative_product_equals_one`.

Exhaustive weak-composition enumeration covers every nonzero binary table with total count $1 \le n \le 8$: 12,869 tables and 308,856 coordinates. No nonempty product-one expression occurs below $n = 8$. Exactly 16 occur at $n = 8$; all have support size five, five canonical terms, and are net unique-one or net unique-two atoms. Thus $n = 8$ and support five are minimal only inside this exhausted binary domain.

5 Bounded executable refinement

5.1 Preflight before powering

For each canonical term, the Rust route first checks that $e_j = nq_j$ is a nonzero integer. It then computes without exponentiation

$$B = \sum_j |e_j|\big(\mathrm{bits}(\mathrm{num}\,x_j) + \mathrm{bits}(\mathrm{den}\,x_j)\big). \tag{26}$$

The route admits exact powers only under all ceilings in Table 1.

Table 1: Exact-product resource policy.

Limit	Ceiling
Canonical terms in one compared expression	256
Absolute denominator-cleared exponent	16,384
Projected product bits in one expression	262,144
Aggregate projected bits over admitted coordinates	1,048,576
Parser total-count bit ceiling (not a powering permission)	8,192

The preflight uses arbitrary-precision integers only for exponent and projection calculations. An expression over a per-expression limit records `not_compared_per_expression_preflight_limit`. If individually admissible expressions exceed the aggregate ceiling, nonempty expressions record `not_compared_total_preflight_limit`. These are fallbacks, not rejected interval certificates. No rational power is allocated for an unavailable plan.

5.2 Exact comparison and interval consistency

For an admitted plan, the producer computes exact positive rational powers and multiplies them as normalized rationals. It compares the final numerator and denominator. It does not serialize a potentially large product and does not factor integers. A postcondition checks that the final numerator-plus-denominator bit length does not exceed equation (26).

The exact-product and directed-interval results have different semantics. If the exact product is positive, the interval must not lie wholly below zero; if negative, it must not lie wholly above zero; if one, the interval must contain zero. An exact result may refine an unresolved interval, but it never changes the interval-local field.

5.3 Independent verifier

The standard-library Python verifier independently reconstructs all keyed event counts, exact log-linear expressions, the integer Möbius transform, and the product preflight. For an admitted coordinate it independently evaluates the rational powers using `Fraction` and validates the report decision and witness. Independently of that sign path, it proves each directed interval contains the exact-real logarithmic expression using outward integer fixed-point bounds on

$$\log y = 2\sum_{k\geq 0}\frac{z^{2k+1}}{2k+1}, \qquad z=\frac{y-1}{y+1}, \quad 1\leq y<2,$$

with $0 \leq z \leq 1/3$ and tail bound

$$0 \leq R_m \leq \frac{2z^{2m+1}}{(2m+1)(1-z^2)} \leq \frac{9z^{2m+1}}{4(2m+1)}. \tag{27}$$

The two implementations share the mathematical specification but not the arithmetic library. Python and this verifier remain trusted; this is not a formal refinement theorem.

6 Formal and adversarial evidence

6.1 Lean kernel check

The standalone Lean 4.32 module proves seven results: finite signed-power log normalization, the scaled $n^{-1}\log R$ form, positive, negative, and zero equivalences, the explicit reciprocal cancellation

$$\log x + \log(x^{-1}) = 0 \qquad (x>0,\ x\neq 1). \tag{28}$$

and the exact rational identity for all five factors in equation (24). The checker rejects `sorry`, `admit`, `axiom`, and `unsafe`. The reported axiom basis is `propext`, `Classical.choice`, and `Quot.sound`. Lean does not yet formalize the concrete SxPID events, count extractor, lattice table, Rust refinement, or resource behavior. The independent exact-rational and Rust routes bind the checked five-factor identity to the SxPID2 unique-one net atom.

6.2 Bounded exact qualification

Every nonzero binary count table with total at most four yields 494 tables and 11,856 coordinates. The exact route checks 5,280 event-nesting obligations, 5,280 local net identities, 1,482 direct-MI identities, 3,952 component-net identities, and 5,928 zeta reconstructions. The products classify 5,886 exact zeros, 5,762 positives, and 208 negatives. All report term products and exact-product decisions agree with the independent derivation.

The fail-closed exact-product self-test kills six semantic source mutations, eleven resealed certificate mutations, and four structural adversaries, for 21 total. The boundary checker kills a resealed false sign on Counterexample 4.1. The deterministic evolutionary search uses seed `0x5358504944322026`, total 64, population 96, and 96 generations. It evaluates 5,921 distinct tables while searching for a negative informative or misinformative partial atom and finds none. That is negative search evidence, not a theorem of nonnegativity.

7 Five-lens review

Lens	Conclusion
Definition	The event scan uses the shared-exclusions source disjunction and keyed target intersection for four pinned two-source nodes. It does not replace the redundancy union by an intersection and does not import another PID's atoms or axioms.
Proof	Finiteness, positive exact arguments, positive integer n, count weights, and integer Möbius coefficients are explicit. The sign theorem is complete for the fixed empirical coordinate; scientific adequacy is not a premise or conclusion.
Executable refinement	Rust event terms, exact products, report evidence, and a separately implemented Python reconstruction agree on bounded domains. This is strong bounded evidence, not a universal compiler-level refinement theorem.
Numerical/statistical meaning	Exact product comparison removes transcendental sign ambiguity. It does not provide sampling uncertainty, effect stability, or population validity. A tiny exact sign can be statistically unstable.
Adversarial falsification	Exhaustion found the exact counterexample that invalidated the first completeness assumption; mutations challenge events, weights, Möbius signs, comparison, product evidence, and resource traces. Evolutionary failure to find another class of violation remains incomplete evidence.

8 Positive and negative findings

8.1 Positive findings

- Every fixed empirical two-source cumulative and integer-Möbius atom has the exact form $n^{-1} \log R$.
- Exact zero and sign reduce completely to rational comparison with one; no logarithm is evaluated for that decision.
- The live Rust route is resource-bounded before powering and exposes unavailable decisions rather than silently escalating work.
- Independent product reconstruction, rigorous interval containment, Lean algebra, bounded exhaustion, mutations, and evolutionary search provide distinct evidence layers.

8.2 Negative findings and retained limitations

- Empty canonical terms are not a complete zero criterion. Counterexample 4.1 is a concrete negative result and remains in the qualification corpus.
- Exact arithmetic does not imply atom nonnegativity. The small exhaustive corpus contains 208 negative net coordinates.
- Tolerance-only zero classification is avoidable and loses exact information; 5,886 coordinates in the small corpus are exactly zero.
- Exact rational products can be expensive. The bounded route can abstain, and its abstention is not a sign or zero decision.

- The Lean theorem is generic; concrete event extraction and executable refinement are not yet proved end to end in one kernel.
- The scientific choice and interpretation of SxPID, statistical calibration, continuous estimators, and downstream authority remain outside this theorem.
- Exhaustive and evolutionary searches are bounded. Their negative findings cannot replace general mathematical proofs.

9 Reproduction and evidence map

From the repository root:

```
python3 audit/tools/certified-sxpid/scripts/check-exact-products.py
python3 audit/tools/certified-sxpid/scripts/check-exact-products-self-test.py
python3 audit/tools/certified-sxpid/scripts/check-nonsyntactic-zero-boundary.py
python3 audit/tools/certified-sxpid/scripts/challenge-exact-products.py \
  --output audit/evidence/sxpid2-exact-product-evolutionary-challenge.json
python3 scripts/check-lean-exact-log-product.py
scripts/check-exact-log-product-sxpid2-pdf.sh --exact
```

The machine-readable receipts are

- `audit/evidence/sxpid2-exact-product-qualification.json`;
- `audit/evidence/sxpid2-exact-product-mutation-suite.json`;
- `audit/evidence/sxpid2-exact-product-nonsyntactic-zero-boundary.json`;
- `audit/evidence/sxpid2-exact-product-evolutionary-challenge.json`; and
- `audit/evidence/sxpid2-exact-product-lean-check.json`.

10 Conclusion

The exact product theorem is elementary but decisive: once the finite empirical SxPID2 semantics are fixed, exact sign is rational arithmetic. The important methodological advance is not merely the theorem; it is the retained falsification of the first incomplete zero witness and the strict separation of interval-local evidence from exact multiplicative evidence. The repaired route strengthens numerical assurance while leaving every statistical, scientific, and downstream claim at its proper boundary.

References

[1] A. Makkeh, A. J. Gutknecht, and M. Wibral. Introducing a differentiable measure of pointwise shared information. *Physical Review E*, 103:032149, 2021.

[2] P. L. Williams and R. D. Beer. Nonnegative decomposition of multivariate information. *arXiv:1004.2515*, 2010. This reference supplies historical PID context only; its $I_{\min}$ measure is not used by the exact SxPID2 theorem.